

Detection Engineering for Data Loss Prevention: Designing Splunk-Based Alerts and Response Plans for Data Exfiltration Incidents

Pedram Shahbazi, Robert (Ash) Drake,

With collaboration of Samsung Austin Semiconductor (Engineer Eric Phillpott)

Project (CS-6399-D10)

Dr. Erdoğan Doğdu, Dr. Roya Choupani

Progress Report # 1

Master of Computer Science

Department of Computer Science

Angelo State University

Spring 2026

Problem Statement

Data exfiltration incidents, whether driven by external attackers or malicious insiders, represent a critical threat to the confidentiality and integrity of organizational information assets. As enterprises increasingly adopt cloud services, remote work models, and heterogeneous infrastructures, data can leave corporate boundaries through a wide variety of channels, including web uploads, cloud storage platforms, email, collaboration tools, and covert network protocols. Traditional Data Loss Prevention (DLP) solutions typically focus on content inspection and policy enforcement within a single channel, which often leads to fragmented visibility and an overwhelming number of isolated alerts.

Security Information and Event Management (SIEM) platforms such as Splunk are designed to aggregate logs and events from multiple security and IT systems, offering a centralized view of activity across the environment. However, many organizations underutilize this aggregated data for DLP-specific detection engineering and lack well-defined response processes tailored to data exfiltration scenarios. The core problem addressed by this project is the absence of a structured, Splunk-based framework that uses aggregated telemetry to detect diverse data-exfiltration behaviors and couples these detections with clear, actionable incident response plans.

This project seeks to design, implement, and evaluate a detection-engineering solution for DLP scenarios that operates on aggregated data within Splunk. The solution will specify detection use cases for multiple exfiltration vectors, define the required data and system architecture, implement correlation and alerting logic, and document response playbooks that guide Security Operations Center (SOC) analysts from alert generation through to containment and recovery.

Related Work

Literature References

Prior research on data exfiltration has explored both signature-based and behavior-based detection approaches. Network-centric studies analyze flow statistics, traffic volumes, and protocol-specific features to identify anomalous outbound behavior indicative of data theft. Host-centric and application-centric works focus on access patterns, file operations, and process behaviors. More recent research has applied statistical models and machine learning to detect deviations from established baselines, particularly for low-and-slow exfiltration attempts that evade simple volume thresholds.

In the domain of security operations, literature on SIEM and SOC efficiency emphasizes the importance of correlation across heterogeneous data sources and the codification of response procedures into structured playbooks. These works highlight that effective detection is not only a matter of signal quality but also of how well alerts integrate into the operational workflows of analysts, including triage, enrichment, and remediation steps.

Other Products and Projects

Commercial DLP solutions and Cloud Access Security Brokers (CASBs) provide extensive policy-based capabilities for monitoring and controlling data movement across endpoints, email, web gateways, and cloud services. They often include pre-built policies for sensitive data types and regulatory categories, as well as dashboards and incident views for compliance reporting. However, these products typically generate alerts within their own consoles and may lack deep, out-of-the-box correlation with other infrastructure and security logs, leading to siloed incident handling.

SIEM and Extended Detection and Response (XDR) platforms commonly offer predefined rules and content packs for detecting suspicious outbound traffic patterns, abnormal cloud activity, or signs of DNS tunneling. These content packs can accelerate deployment but usually require customization to reflect an organization's specific environment, threat model, and risk tolerance. Furthermore, many vendor-provided SOC playbooks focus on general incident response workflows, such as those for malware or account compromise, and treat data exfiltration as a subcase without fully leveraging DLP-specific context.

Critical Comparison with the Proposed Design

The proposed project design differs from existing work and products in three main respects. First, it explicitly frames DLP for data exfiltration as a detection-engineering problem within Splunk, rather than as a feature of a standalone DLP solution. This perspective emphasizes cross-channel correlation, use-case-driven feature design, and iterative tuning based on SOC feedback. Second, while vendor content and academic models often address specific channels (e.g., network flows, DNS tunneling, or email misuse), this project focuses on a coherent set of use cases that span multiple channels and tie them together through aggregated analytics. Third, the project tightly couples detection logic with concrete incident response playbooks, bridging the gap between analytics and operations, and producing artefacts that can be directly consumed by SOC teams.

Requirements

Functional Requirements

The system shall:

1. Data Ingestion and Normalization

- Ingest network, proxy, or firewall logs providing source, destination, and byte-count fields.
- Ingest DNS logs capturing query names, query types, responses, and client identifiers.
- Ingest SaaS and office-suite audit logs for email and file operations, including events such as message send, forwarding, and mailbox export.
- Ingest DLP or related security logs that report policy violations, rule matches, and actions taken.

2. Detection Use Cases

- Define at least four data-exfiltration use cases, covering scenarios such as:
- High-volume outbound transfers from internal hosts or users.
- Transfers to unsanctioned or rare external domains, including cloud storage providers.
- Email-based exfiltration via bulk sending, mass forwarding, or mailbox/PST export.
- DNS-based exfiltration patterns characterized by abnormal query volumes, lengths, or structures.

3. Detection Logic Implementation

- Implement Splunk searches and correlation rules that:
- Aggregate events over configurable time windows by user, host, or other entity.
- Compute features such as total bytes, event counts, destination rarity, and deviation from historical baselines.
- Generate alerts or notable events with contextual details, including entities involved, channel used, and supporting evidence.

4. Response Playbooks

- For each detection use case, define an incident response playbook that includes:
- Initial validation steps to confirm or refute the suspected exfiltration.
- Enrichment actions, such as retrieving asset information, user history, and related alerts.

- Containment and remediation actions, such as blocking destinations, isolating hosts, or suspending accounts.
- Escalation paths, communication guidelines, and documentation of the incident.

5. Evaluation and Reporting

- Execute controlled tests using synthetic or lab-generated data to verify that detections respond correctly to simulated exfiltration scenarios.
- Evaluate the rate of false positives against benign traffic scenarios.
- Compile results, lessons learned, and recommendations into a structured report.

In summary Functional requirements will include: Generating alerts by system; Receiving alerts by users; System detection of large volume outbound from the sources of Web proxy logs, Gateway firewalls, Emails, ...

Non-Functional Requirements

The system shall exhibit the following non-functional properties:

- **Performance:** Detection searches must complete within an acceptable time frame for operational use, considering the expected data volume and time window.
- **Scalability:** The design must remain viable as data volumes grow, using efficient aggregations and avoiding unnecessary complexity in SPL searches.
- **Reliability:** The detection logic and playbooks must function consistently, with clear error-handling strategies in case of missing data or partial ingestion.
- **Explainability:** Each detection rule must be transparent, with documented logic, thresholds, and potential limitations that can be understood by SOC analysts.
- **Maintainability:** All rules, playbooks, and configurations must be modular, version-controlled, and accompanied by documentation to facilitate future updates and extensions.
- **Compliance and Privacy:** The project must respect applicable privacy and compliance constraints, relying on anonymized or synthetic data when necessary.
- Under 500MB/day ingestion (stay in license)
- 100% uptime (<5min ingestion delay)

Constraints

Key constraints affecting the design include:

- Limited access to real production security logs at Samsung Austin, necessitating the use of synthetic or test datasets for certain scenarios.
- Limited project duration, which constrains the number of use cases and the depth of advanced analytics that can be implemented.
- Potential absence of full SOAR integration, requiring that some playbooks remain conceptual or semi-manual.
- Restricted computational resources in the lab environment, influencing the complexity and frequency of scheduled searches.

System Design

Architectural Overview

The system is organized into four main layers:

1. Data Source Layer

- Comprises network/proxy/firewall devices, DNS infrastructure, SaaS and email platforms, and DLP tools.
- Each source produces logs related to connectivity, name resolution, file access, email operations, and policy violations.

2. Ingestion and Normalization Layer (Splunk Core)

- Responsible for receiving data from the various sources, assigning appropriate indexes and source types, and performing field extractions.
- Establishes a common set of fields (e.g., timestamp, user, host, src_ip, dest_ip, url, domain, bytes, action) to enable correlation across sources.
- Optionally leverages data models that abstract events into logical domains such as Web, Network Traffic, Endpoint, Email, and DLP.

3. Analytics and Detection Layer

- Implements feature extraction, baselining, and detection logic using SPL.
- Aggregates data per entity and time window to compute metrics such as total bytes outbound, number of external domains contacted, frequency of specific event types, and statistical deviations from historical behavior.
- Encodes use-case-specific rules and correlation searches that identify candidate exfiltration events.

4. Alerting and Response Layer

- Generates alerts or notable events when detection rules are triggered.
- Attaches relevant context to alerts, such as involved entities, supporting events, and basic risk scoring.

- Provides entry points into documented incident response playbooks, which may be executed manually by analysts or partially automated.

Logical Modules

The system is decomposed into the following logical modules:

1. Data Ingestion and Normalization Module

- Configures inputs, indexes, and source types.
- Implements field extractions and lookups to ensure consistent representation of user identities, assets, and destinations.

2. Feature Extraction Module

- Defines saved searches or macros to compute per-entity aggregates and derived features.
- Maintains baseline statistics for normal behavior where applicable, such as average daily outbound volume for hosts or users.

3. Detection Module

- Implements SPL searches for each exfiltration use case.
- Encodes rule logic, thresholds, and correlation conditions (e.g., combination of high volume and unsanctioned destination).
- Produces structured outputs suitable for alerting and downstream analysis.

4. Alerting and Enrichment Module

- Converts detection results into alert entities with severity and classification.
- Enriches alerts with contextual fields drawn from asset and identity repositories, or from prior alerts and incidents.

5. Response Playbooks Module

- Stores playbook definitions and their decision trees.
- Guides analysts through triage, investigation, containment, and closure steps.

6. Evaluation and Reporting Module

- Records test scenarios and outcomes.
- Generates summary statistics and visualizations for the final report and presentation.

Data and Conceptual Model

Although Splunk is not a relational database, the conceptual data model for this project includes:

- **User Entity**
 - Attributes: user_id, department, role, privilege level, typical working hours.

- **Asset Entity (Host/Endpoint/Server)**
- Attributes: asset_id, hostname, IP address, business function, criticality rating.
- **Event Entity**
- Attributes: timestamp, source_type, user, host, src_ip, dest_ip, domain/url, bytes, action, channel (e.g., web, email, DNS, DLP).
- **Alert Entity**
- Attributes: alert_id, detection_use_case_id, severity, status, associated events, analyst owner, timestamps for creation and closure.

Auxiliary lookup structures will define:

- Sanctioned and unsanctioned domains or services.
- Mapping of users to departments and roles.
- Classification of assets (e.g., endpoints, file servers, critical servers).

Technology and Modules

The implementation will employ:

- Splunk Enterprise as the core platform for data ingestion, search, and visualization.
- Optional Splunk applications for security analytics and content, if available, to streamline data modeling and enrichment.
- A scripting environment (e.g., Python) for generating synthetic logs and performing auxiliary data manipulation or analysis.
- Version control (e.g., Git) to manage SPL searches, configuration artefacts, and documentation.

Test Plan

Test Objectives

- Validate that each detection rule correctly identifies its targeted exfiltration scenario.
- Assess the rate and nature of false positives under benign conditions.
- Confirm that response playbooks provide clear and actionable guidance to analysts.

Test Types

1. Unit-Level Detection Tests

- Construct small, controlled datasets for each use case where the expected number and nature of alerts are known.

- Execute detection searches and compare outputs against expectations.

2. Scenario-Based Tests

- Simulate end-to-end exfiltration scenarios (e.g., user uploading a large archive to an unsanctioned domain, bulk email forwarding to external recipients, or DNS tunneling patterns).
- Verify the complete chain: data ingestion → detection → alert generation → engagement of playbook steps.

3. Baseline and False-Positive Tests

- Apply detection rules to benign datasets representing normal user activity.
- Record and analyze alerts generated, adjusting thresholds and filters where necessary.

4. Performance Tests

- Measure query execution times for typical look-back periods and data volumes.
- Identify bottlenecks and optimize searches or indexing strategies if required.

Test Artefacts

- A catalog of test cases describing input conditions, expected results, and actual outcomes.
- Screenshots or exports of detection results and alert views.
- Logs of tuning changes and their impact on detection quality.

Risk Plan

Identified Risks and Mitigations

R1: Limited Realistic Data

- *Risk:* Lack of diverse, real-world logs may reduce the realism of detections and evaluations.
- *Mitigation:* Use carefully designed synthetic datasets informed by known exfiltration behaviors; clearly document limitations and assumptions.

R2: High False-Positive Rate

- *Risk:* Overly simplistic detection rules may generate excessive alerts.
- *Mitigation:* Employ baselining, per-entity thresholds, contextual filtering, and iterative tuning based on test results.

R3: Time Constraints

- *Risk:* Insufficient time to implement all planned use cases or advanced analytics.

- *Mitigation:* Prioritize a core subset of high-value use cases; treat additional ideas as future work; maintain a clear scope boundary.

R4: Tooling or Integration Limitations

- *Risk:* Inability to integrate certain data sources or response systems in the lab environment.
- *Mitigation:* Provide conceptual integration designs and focus implementation on available components; document assumptions and gaps.

R5: Complexity of Detection Logic

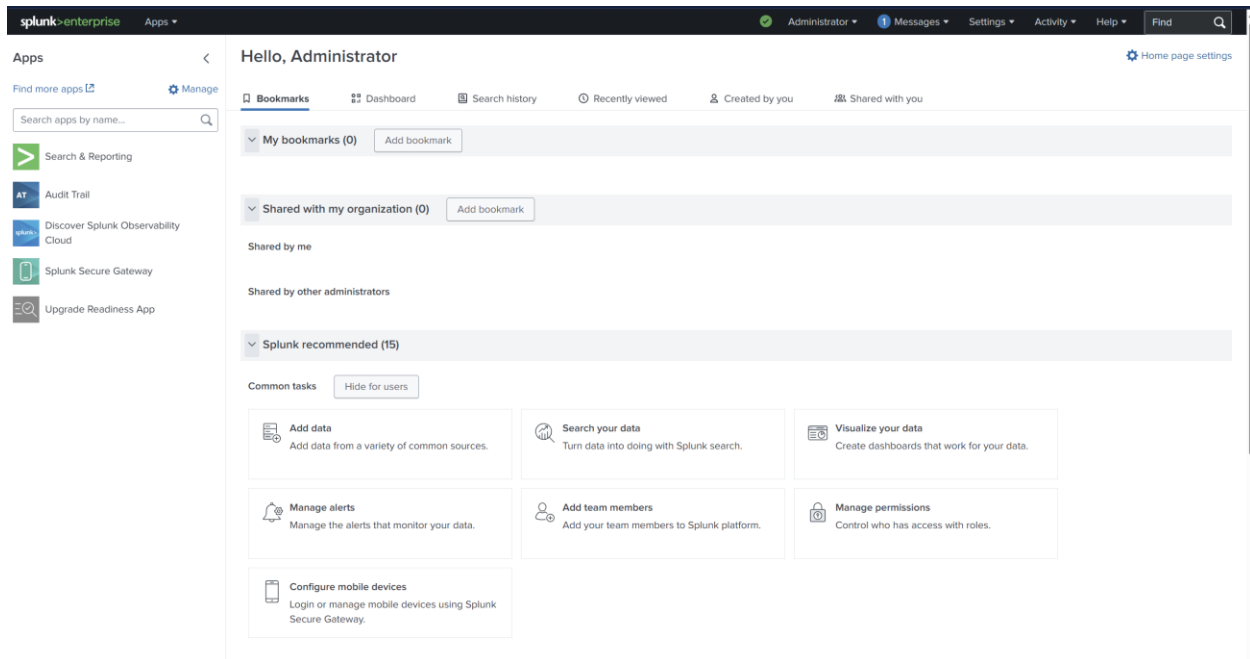
- *Risk:* Overly complex rules may be difficult to understand and maintain.
- *Mitigation:* Favor interpretable logic and incremental complexity; thoroughly document rule behavior and rationale.

Implementation Status

Phased Implementation

1. Phase 1 – Environment Setup and Requirements Realization (Weeks 1–3)

- Installed Splunk environment and configuring it for required tasks.



- Onboarding representative dataset/s (e.g., network logs) and validating field extractions providing by Samsung Austin Semiconductor (Thanks to Eng. E. Phillpott).
- Finalizing detailed requirements and use-case specifications.

2. Phase 2 – Data Model and Feature Design (Weeks 3–4)

- Define the conceptual data model and mapping from raw logs to logical entities.
- Implement feature extraction searches for core metrics (e.g., per-host outbound bytes, domain contact frequency).

3. Phase 3 – Detection Implementation (Weeks 5–7)

- Implement SPL rules for high-volume outbound and unsanctioned destination use cases.
- Extend detection logic to email-based and DNS-based exfiltration scenarios.
- Conduct preliminary validation on small test datasets.

4. Phase 4 – Playbooks and Alerting Integration (Weeks 8–9)

- Design and document response playbooks for each use case.
- Integrate detection outputs into alerting mechanisms and views for analysts.

5. Phase 5 – Testing, Tuning, and Evaluation (Weeks 10–12)

- Run controlled benign and malicious test scenarios; collect outcomes.
- Tune thresholds and logic to manage false positives and false negatives.
- Document evaluation results and preliminary conclusions.

6. Phase 6 – Documentation and Finalization (Weeks 13–14)

- Prepare the final project report and supporting materials.
- Refine diagrams, tables, and examples; ensure reproducibility of key steps.

Project timeline:

Task Breakdown

Representative tasks include:

- T1: Configuring data inputs, indexes, and field extractions.
- T2: Define use-case templates and threat models.
- T3: Implement feature extraction searches and baselines.
- T4: Implement correlation searches for each detection use case.
- T5: Draft and refine incident response playbooks.
- T6: Develop synthetic data and test scenarios.
- T7: Execute tests and collect metrics.
- T8: Compile documentation, diagrams, and final report sections.

Completion status

- Completed registering and installation of Splunk Enterprise
- Getting Splunk environment ready for loading logs and other data

Project Timeline

A representative semester-length timeline, with dependencies, is as follows:

- **Weeks 1–2**
 - Finalization of requirements and use-case definitions (depends on initial background study).
 - Initial Splunk environment setup and configuration.
- **Week 3**
 - Completion of data ingestion for at least one core log type (dependency for feature design).
 - Draft of conceptual data model and mapping.
- **Weeks 4–5**

- Implementation of feature extraction searches and baselines (depends on data ingestion).
- Implementation of detection rules for high-volume outbound and unsanctioned destination use cases.
- **Weeks 6–7**
 - Implementation of detection rules for email-based and DNS-based exfiltration.
 - Initial drafting of response playbooks (depends on having at least preliminary detections).
- **Weeks 8–9**
 - Refinement and finalization of response playbooks.
 - Development of synthetic datasets and test scenarios (depends on defined use cases and detections).
- **Weeks 10–11**
 - Execution of test plan, including scenario-based and false-positive testing (depends on test data and detections).
 - Tuning of detection rules and documentation of results.
- **Weeks 12–14**
 - Preparation of the final project report, including design, implementation, and evaluation chapters.
 - Creation of supporting diagrams, tables, and presentation materials.

Conclusion

This project design presents a formal blueprint for developing a Splunk-based detection-engineering solution focused on data-exfiltration scenarios relevant to DLP. It articulates the problem context, captures functional and non-functional requirements, and specifies an architecture that leverages aggregated telemetry and structured analytics. The design decomposes the system into logical modules, outlines a realistic implementation and testing strategy, and accounts for key risks and constraints. By following this design, the project is positioned to produce both technically robust detection content and operationally meaningful response playbooks tailored to SOC workflows.

References

- Splunk Threat Research Team. “Data Exfiltration Detections: Threat Research Release, June 2021.” 2021.
- Splunk. “Detecting Data Exfiltration Activities.” Splunk Lantern.
- Splunk. “Machine Learning in Security: Detect DNS Data Exfiltration Using Deep Learning.” 2022.

- Splunk. “Machine Learning in Splunk Enterprise Security: Unleashing Hidden Threats.” 2025.
- SearchInform. “SIEM Workflow Automation: Streamlining Incident Response.” 2025.
- Radiant Security. “SOC Playbook Examples for Real-World Cyber Threats.” 2026.
- Hamdan, M. “Data Exfiltration Explained: How to Use Splunk and Wireshark to Detect It.” 2025.
- Splunk Lantern. “Data Loss Prevention Data.” 2025.
- PuppyGraph. “How to Build a SIEM System: Architecture & Tools.” 2026.